

Tema 10: Implementació de llistes

1. Estructura interna: Doble enllaç i Sentinelles

A diferència del vector, una llista no guarda els elements junts en memòria. Cada element està en un **Node** (o **Item**) independent que sap qui té davant i qui té darrere.

Nodes Sentinel·la (**iteminf** i **itemsup**)

Aquesta implementació utilitza dos nodes especials que **sempre existeixen**, encara que la llista estigui buida:
- **iteminf**: El node “fictici” inicial. El seu **next** apunta al primer element real. - **itemsup**: El node “fictici” final. El seu **prev** apunta a l’últim element real.

Avantatge: No hem de comprovar mai si un punter és **nullptr** en fer insercions o esborrats, simplificant molt el codi. - **Truc per als exercicis**: Com que els sentinel·les sempre hi són, el primer element real és **iteminf.next** i l’últim és **itemsup.prev**. Pots accedir-hi directament per fer operacions com **swapFirstLast**.

```
template <typename T>
class List {
    struct Item {
        T value;
        Item *next, *prev;
    };
    int _size;
    Item iteminf, itemsup; // Nodes reals (no punters)
public:
    // ...
};
```

2. El motor: Inserir i Esborrar en $\Theta(1)$

La llista és l’estructura ideal per inserir/esborrar en qualsevol punt si ja tenim la posició. Només cal “recosir” els punters.

Inserir un element (**insertItem**)

1. Creem el nou node.
2. El connectem amb el seu següent i anterior.
3. Actualitzem els punters dels veïns perquè apuntin al nou node.

```
void insertItem(Item *pitemprev, const T &value) {
    Item *pitem = new Item;
    pitem->value = value;

    pitem->next = pitemprev->next;
    pitem->next->prev = pitem;
    pitem->prev = pitemprev;
    pitemprev->next = pitem;
```

```

    _size++;
}

```

Com moure nodes (la regla dels 4 punters)

En molts exercicis (com `moveToEnd` o `moveSecondToLast`), el Jutge prohibeix intercanviar els `.value`. Has de moure el node físicament: 1. **Disconnectar**: Uneix el veí anterior amb el següent (`p->prev->next = p->next` i `p->next->prev = p->prev`). 2. **Reconnectar**: Insereix el node a la nova posició ajustant els seus nous `next` i `prev` i els dels seus nous veïns.

Si moure un node implica que aquest sigui veí d'ell mateix (casos de 2 elements), ves amb compte de no perdre el punter abans d'hora! Ús de punters temporals és recomanat.

3. Iteradors: El pont cap a les dades

Com que els nodes estan dispersos, no podem usar `[i]`. Usem **iteradors**, que actuen com un punter intel·ligent que sap com moure's per la llista.

- **begin()**: Apunta al primer element real (`iteminf.next`).
- **end()**: Apunta al node sentinella final (`itemsup`). Mai s'ha de desreferenciar!

```

class iterator {
    List *plist;
    Item *pitem;
public:
    T& operator*() { return pitem->value; }
    iterator operator++() { // Preincrement (++it)
        pitem = pitem->next;
        return *this;
    }
};

```

Iteradors Circulars: Si un exercici demana que la llista sigui circular, l'`operator++` de l'últim node no ha d'anar a `end()`, sinó a `begin()`. En la nostra implementació amb sentinelles, això significa saltar de `itemsup` a `iteminf.next`.

::linkedlistviz

4. Gestió de memòria: La Regla dels Tres

Com que gestionem nodes amb `new`, hem de ser molt curosos: 1. **Destructor**: Ha d'esborrar TOTS els nodes (usant `removeItem` en bucle). 2. **Constructor de còpia**: Ha de crear nodes nous i copiar els valors. 3. **Operador d'assignació**: Netejar la llista actual i copiar la nova.

A l'assignació `l1 = l2`, sempre hem de comprovar l'**auto-assignació** (`this != &l1`) per no esborrar les dades que volem copiar.

Vector vs Llista: Quan usar quina?

Característica	Vector (<code>std::vector</code>)	Llista (<code>std::list</code>)
Accés aleatori <code>[i]</code>	$\Theta(1)$	$\Theta(n)$ (cal recórrer)
Inserir al final	$\mathcal{O}(1)$ amortitzat	$\Theta(1)$
Inserir al principi	$\Theta(n)$	$\Theta(1)$
Inserir al mig (amb <code>it</code>)	$\Theta(n)$	$\Theta(1)$

Característica	Vector (<code>std::vector</code>)	Llista (<code>std::list</code>)
Memòria	Bloc contigu (més ràpid per la CPU)	Nodes dispersos (més overhead)

Operacions amb Iteradors

Operació	Codi	Explicació
Inserir	<code>it = l.insert(it, val)</code>	Insereix abans de <code>it</code> .
Esborrar	<code>it = l.erase(it)</code>	Esborra <code>it</code> i retorna el següent .
Recórrer	<code>for(auto it=l.begin(); it!=l.end(); ++it)</code>	El patró estàndard.